# Day 44: DFS (Depth First Search)

🔹 1. DFS (Depth First Search)

👉 Concept

DFS me hum depth me jaate hain first, matlab:

Ek node se start karo

Uske neighbor pe jao

Phir uske neighbor pe… jab tak end na mile

Fir backtrack karo

📌 Exactly like: Tree traversal (Preorder)

---

🔹 DFS Example

Graph:

0 → 1 → 2
↓
3

DFS traversal (starting from 0):

0 → 1 → 2 → (back) → 3

---

🔹 2. DFS using Recursion (Java Code)

```
1. import java.util.*;

class GraphDFS {
    int V;
    ArrayList<ArrayList<Integer>> adj;

    GraphDFS(int V) {
        this.V = V;
        adj = new ArrayList<>();

        for (int i = 0; i < V; i++)
            adj.add(new ArrayList<>());
    }

    void addEdge(int u, int v) {
        adj.get(u).add(v);
        adj.get(v).add(u); // Undirected graph
    }

    void dfs(int node, boolean[] visited) {
        visited[node] = true;
        System.out.print(node + " ");
```

```
        for (int neighbor : adj.get(node)) {
            if (!visited[neighbor]) {
                dfs(neighbor, visited);
            }
        }
    }

    public static void main(String[] args) {
        GraphDFS g = new GraphDFS(5);

        g.addEdge(0, 1);
        g.addEdge(0, 3);
        g.addEdge(1, 2);
        g.addEdge(3, 4);

        boolean[] visited = new boolean[5];

        g.dfs(0, visited);
    }
}
```

---

- Output

0 1 2 3 4

---

- 3. DFS using Stack (Iterative)

```
1. void dfsIterative(int start) {
    boolean[] visited = new boolean[V];
    Stack<Integer> stack = new Stack<>();

    stack.push(start);

    while (!stack.isEmpty()) {
        int node = stack.pop();

        if (!visited[node]) {
            visited[node] = true;
            System.out.print(node + " ");
        }

        for (int neighbor : adj.get(node)) {
            if (!visited[neighbor]) {
                stack.push(neighbor);
            }
        }
    }
}
```

- 4. Important DFS Graph Problems

Ye sab interview me frequently aate hain 👇

## ✅ 1. Number of Connected Components

👉 Idea:

Har node pe DFS lagao

Count increase karo jab new unvisited node mile

```
 int countComponents() {
    boolean[] visited = new boolean[V];
    int count = 0;

    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            dfs(i, visited);
            count++;
        }
    }
    return count;
}
```

## ✅ 2. Cycle Detection (Undirected Graph)

```
 boolean dfsCycle(int node, int parent, boolean[] visited) {
    visited[node] = true;

    for (int neighbor : adj.get(node)) {
        if (!visited[neighbor]) {
            if (dfsCycle(neighbor, node, visited))
                return true;
        } else if (neighbor != parent) {
            return true;
        }
    }
    return false;
}
```

## ✅ 3. Path Exists (Source → Destination)

```
boolean hasPath(int src, int dest, boolean[] visited) {
    if (src == dest) return true;

    visited[src] = true;

    for (int neighbor : adj.get(src)) {
        if (!visited[neighbor]) {
            if (hasPath(neighbor, dest, visited))
                return true;
        }
    }
    return false;
}
```

## ✅ 4. Flood Fill (Matrix DFS)

👉 Used in:

Image processing

```
Island problems
 void dfs(int[][] grid, int r, int c, int color) {
    int n = grid.length, m = grid[0].length;

    if (r < 0 || c < 0 || r >= n || c >= m || grid[r][c] != 1)
        return;

    grid[r][c] = color;

    dfs(grid, r+1, c, color);
    dfs(grid, r-1, c, color);
    dfs(grid, r, c+1, color);
    dfs(grid, r, c-1, color);
}
```

## 🔹 5. BFS vs DFS (Quick Difference)

| **Feature** | **BFS** | **DFS** |
|---|---|---|
| **Data Structure** | Queue | Stack / Recursion |
| **Traversal** | Level-wise | Depth-wise |
| **Shortest Path** | Yes | No |
| **Use Case** | Minimum distance | Backtracking, cycles |

## 🔥 Interview Tips

DFS = Recursion + Backtracking

Always use:

boolean[] visited

Graph problems = mostly DFS / BFS variation